Alana Springer – 3741250

PROJECT REPORT

Software Application and Static Code Analysis Tools

Table of Contents

Abstract 1
Introduction 2
Code Analysis 3
Segment 1 – add_cus.php 3
Segment 2 - add_res.php 4
Segment 3 – staffWel.php 5
Segment 4 - login_cus.php 6
Segment 5 - update-form.php 7
Code Fix 8
add_cus.php 10
add_res.php 10
staffWel.php 11
Login_cus.php 11
update-form.php 12
Discussion 13
References 14
Appendix 15
add_cus.php 15
add_res.php 16
staffWel.php 17
login_cus.php 21
update-form.php 22

Abstract

The pressures of tight development schedules often impede the rigorous security vulnerability checks necessary for software applications. To address this issue, this project aims to develop a software application with minimal bugs and vulnerabilities through the utilization of the static code analysis tool Psalm. The principal objective is to master the use of the Psalm static code analysis tool, which will effectively enhance the software application and eliminate any potential vulnerabilities that may have eluded the developer. The Psalm tool successfully detected 150 exploitable vulnerabilities within the developed software application, which could adversely impact its functionality in the future. These vulnerabilities have been scrutinized and categorized in the *Code Analysis* segment of this paper. Additionally, the *Code Fix* section of this paper meticulously documents the alterations made to eliminate the vulnerabilities detected by the tool.

Introduction

Static Code Analysis, also referred to as Source Code Analysis, involves examining the source code of a software application without actually running it. This type of analysis falls under the category of white box testing since it inspects the internal structure of the code of the software system and is typically carried out during the implementation phase of the Security Development Lifecycle (Dewhurst, 2017). To perform these this analysis, developers use a static code analyzer, which is defined by the National Institute of Standards and Technology as "a tool that analyses source code without executing the code." These analyzers evaluate different parts of source code to uncover low-quality coding practices that may result in vulnerabilities and provide feedback to the developers (Automation Support for Security: Software Vulnerability Management, 2020)

Static code analysis can be seen as one of the most important procedures done to discover bugs in the early stages of a software's system development. Mark Harman, a professor of software engineering at University College London, states that "Source code analysis and manipulation will always be important. As long as there is computation, there will always be new notation that will be used as source code". The demand for web applications with more creative and intricate features is rapidly increasing, which results in developers having less time to complete these projects. Consequently, this time pressure can lead to the introduction of bugs and security vulnerabilities (Nunes, et al., 2018). Static code analysis heavily minimizes these issues and the cost that may come with such issues.

Psalm is an open-source Static Code Analysis tool for PHP. It is designed to work with PHP codebases of all types, ranging from small to large projects, complex applications, modern or outdated codebases that have been developed years ago. Psalm provides a range of features such as error detection, type inference, and code suggestions which can help developers catch errors and improve the overall quality of the software application (Psalm, n.d.).

Code Analysis

Segment 1 – add_cus.php

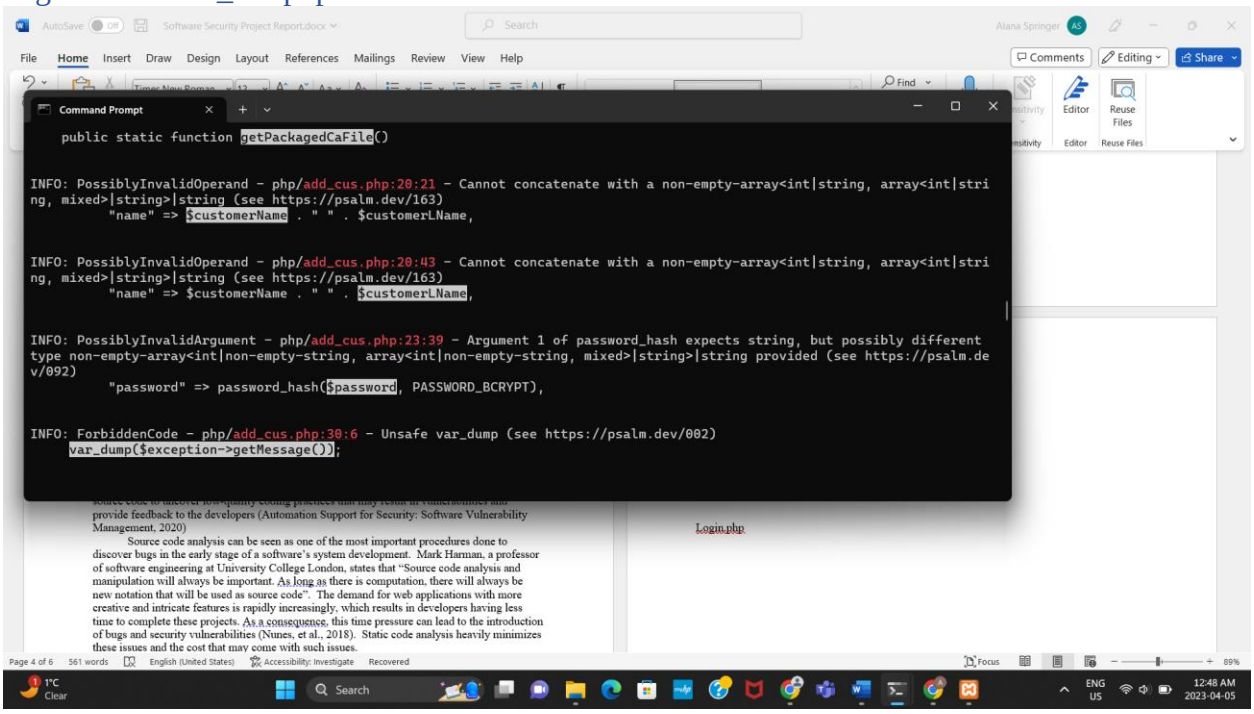


Figure 1: The static code analysis tool shows three types of vulnerability errors.

The type of vulnerability caused by the “PossiblyInvalidOperand” error is known as improper data sanitization, this means the $customerName and customerLname variables were not properly validated or sanitized before being used in the string concatenation operations, which can lead to serious security risks.

The error labeled as "PossiblyInvalidArgument" can lead to incorrect password hashing or unexpected behavior. In cases where the $password variable does not contain a string data type, it can cause problems with the password_hash function, which requires a string as its first argument. This issue can potentially result in the generation of weak or insecure password storage, thus making the software application vulnerable to attacks.

The error labeled as "ForbiddenCode” speaks about the var_dump function. This is considered unsafe because it can potentially reveal sensitive information about the application and server to an attacker.

Segment 2 - add_res.php

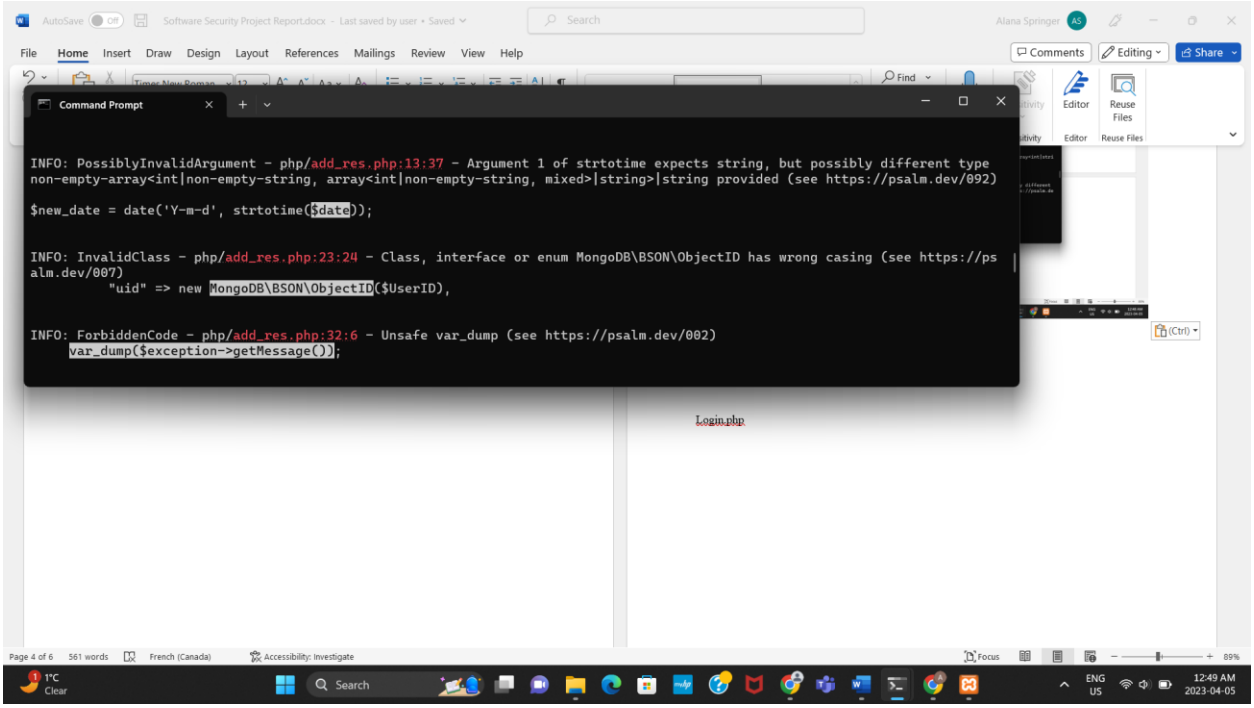


Figure 2 : The static code analysis tool shows three types of vulnerability errors.

The error message “PossiblyInvalidArgument” in Psalm indicates that there is a potential issue with the argument being passed to the **strtotime** function. The error message indicates that the argument passed to **strtotime** may not be a string, but instead may be an array or some other type. If the $date variable is not sanitized properly or if it is manipulated by an attacker, they could potentially pass a malicious input that could cause the strtotime function to behave unexpectedly, leading to incorrect results and potentially disrupting reservation timings in the system.

The "InvalidClass" error in Psalm indicates that the MongoDB\BSON\ObjectID class name, which should be in lowercase.

The error labeled as "ForbiddenCode” speaks about the var_dump function. This is considered unsafe because it can potentially reveal sensitive information about the application and server to an attacker.

Segment 3 – staffWel.php

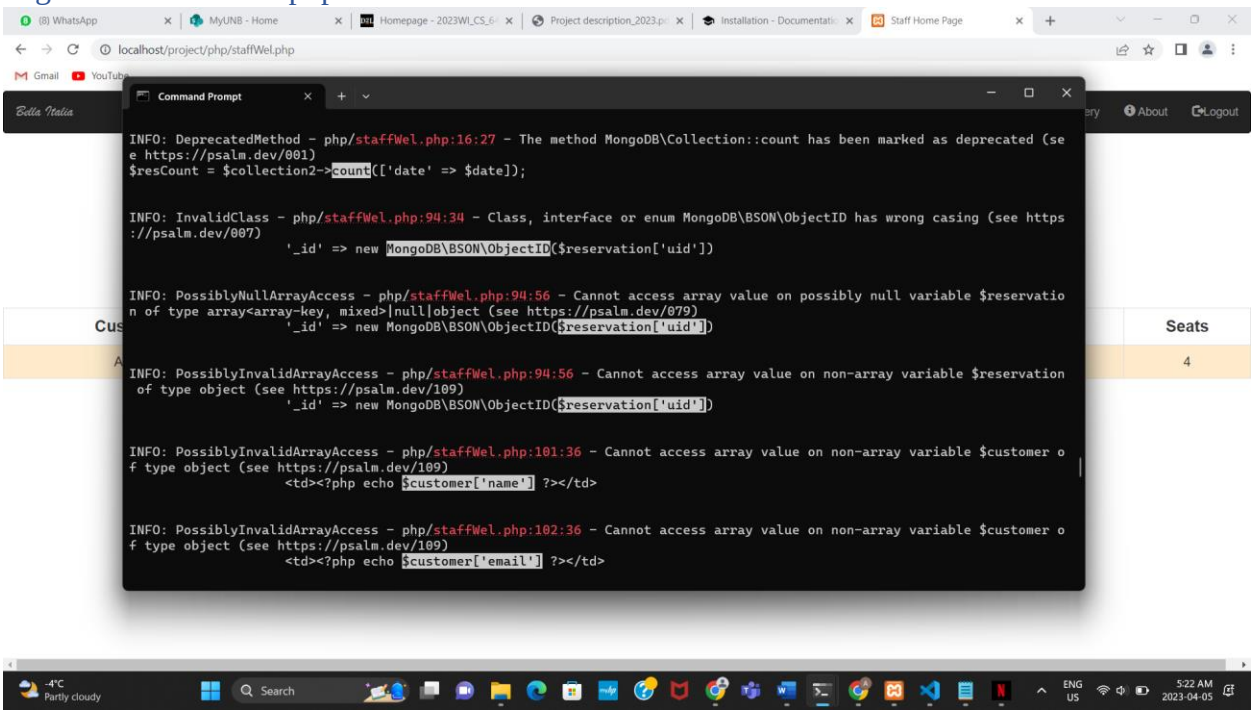


Figure 3: The static code analysis tool shows four types of vulnerability errors.

The error "DeprecatedMethod" indicates that the method count used in the line of code is deprecated, which means that it is no longer recommended to use it because it may not be supported in future versions of the software.

This "InvalidClass" error indicates that the casing of the class name MongoDB\BSON\ObjectID is incorrect. It should be written with a lowercase "o" in ObjectID, but in the code it is written with an uppercase "O" in ObjectID.

The Psalm error "PossiblyNullArrayAccess" means that the code is trying to access a possibly null array. In other words, there is no check done on the variable $reservation to make sure it is actually an array or has a value, which would result in an error when trying to access the 'uid' key.

The "PossiblyInvalidArrayAccess" errors indicate that the code is attempting to access array values on non-array variables of type object. Given that the page is a staff page that grants access to all customer information and reservations, an attacker could potentially compromise the system by exploiting the non-array variables, allowing them to gain access to customer information or modify reservations.

Segment 4 - login_cus.php

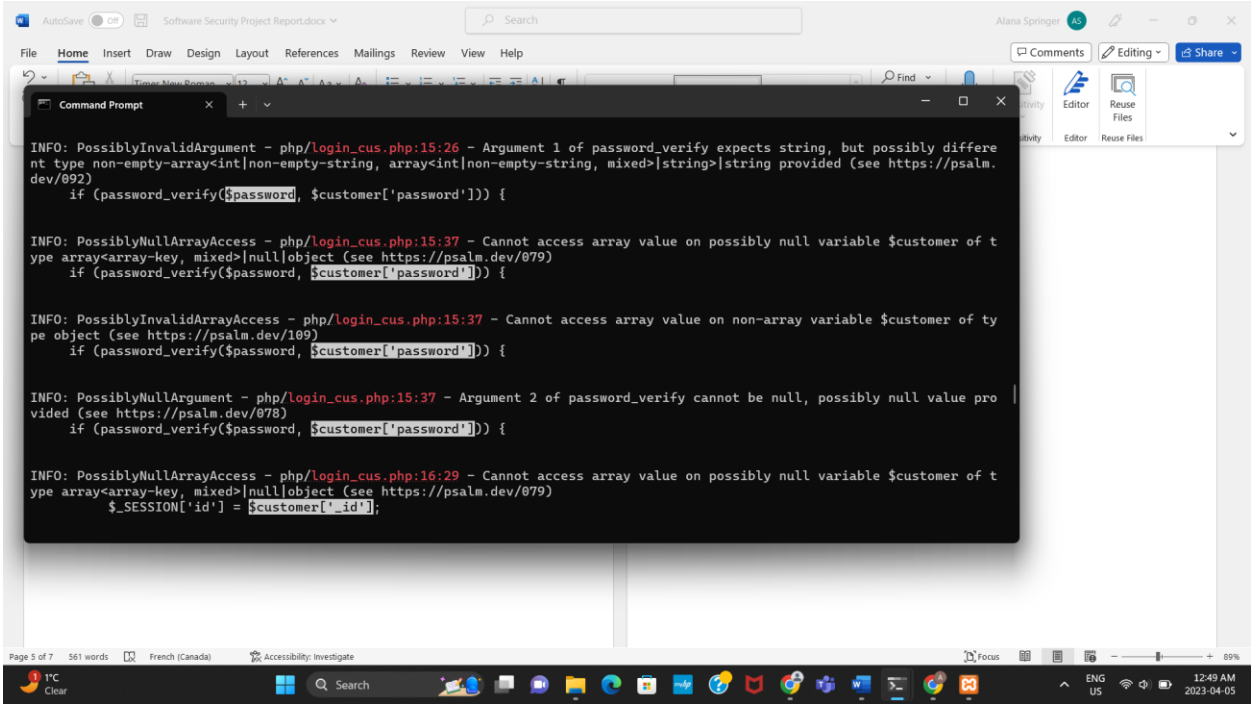


Figure 4: The static analysis showing four different types of vulnerability errors.

The password_verify function is used to verify whether a given password matches the hashed password stored in the database. The "PossiblyInvalidArgument" error suggests that the $password variable is possibly an array instead of a string, which indicates that proper validation checks are not being done on the input data. This can lead to vulnerabilities such as incorrect password verification or authentication bypass attacks. If the $password parameter is not of the expected string type and contains an array or other data type, the function may not work as intended, potentially allowing an attacker to bypass the authentication process and gain unauthorized access to the system.

The Psalm error "PossiblyNullArrayAccess" occurs when there is an attempt to access an array element, but the array variable is possibly null or not initialized. In other words, the code is attempting to access an array element without first checking to make sure that the array exists and has a value.

The Psalm error message "PossiblyInvalidArrayAccess" indicates that the code is trying to access an array value using square brackets on a variable that is not an array. In this segment, the variable $customer is of type object, but the code is trying to access $customer['password'] as if it were an array.

The Psalm error message "PossiblyNullArgument" is indicating that the second argument provided to the password_verify function may be null, which is not allowed by the function. This means that there needs to be a null check before calling the password_verify function to make sure that the $customer['password'] value is not null.

Segment 5 - update-form.php

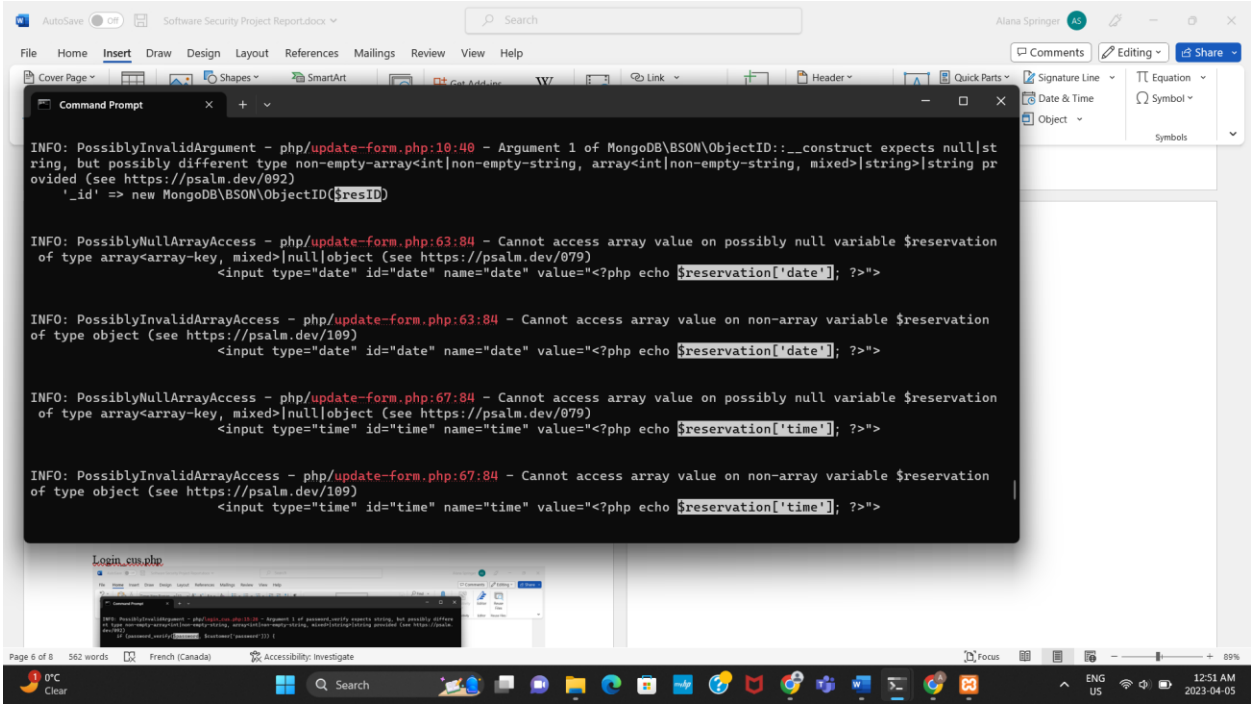


Figure 5: The static analysis showing three different types of vulnerability errors.

The Psalm error message "PossiblyInvalidArgument" indicates that the first argument passed to the MongoDB\BSON\ObjectID constructor is expected to be a string. This means that the $resID variable did not have the necessary checks done to make sure it was a string.

This error "PossiblyNullArrayAccess" means that there is a possibility that the $reservation variable is null or unset, and therefore it may not be possible to access its values. In this segment, the code is trying to access the date key of the $reservation array, but the static analysis tool is warning that it may be null, or the array may not exist, which could lead to runtime errors.

This error "PossiblyInvalidArrayAccess" occurs because the variable $reservation is being treated as an array, but it is actually an object. Therefore, when trying to access the 'date' key as if it were an array index, the static analysis tool reports an error. This error can potentially allow an attacker to manipulate the input data passed to the application in a way that the variable is not an array but is accessed as an array, causing the application to crash or produce unexpected results.

Code Fix

A total of 8 types of errors were found across the pages in various forms.

Types of errors

ForbiddenCode – this speaks about the var_dump which was only there for testing purposes, so it was simply removed.

InvalidClass – The casing of the class name MongoDB\BSON\ObjectID was changed to lower case MongoDB\BSON\objectId

DeprecatedMethod – Since count() would not be supported in future versions of the software, it was changed to *countDocuments()*.

PossiblyInvalidOperand – Changed to a series of if statements validating the first and last name input data.

```
if (is_string($_POST["fname"])) {
    $customerName = $_POST["fname"];
  }
  $customerLName = "";
if (is_string($_POST["lname"])) {
    $customerLName = $_POST["lname"];
  }
```

PossiblyInvalidArgument

Add_cus.php – Error was fixed by ensuring the password is a string.

```
 $password = "";
if (is_string($_POST["password"])) {
    $password = $_POST["password"];
  }
```

Add_res.php – Error was fixed by ensuring the date is a string.

```
if (is_string($_POST["date"])) {
    $date = $_POST["date"];
  }
```

Login_cus.php – Error was fixed by ensuring the password is a string.

```
if (is_string($_POST["password"])) {
    $password = $_POST["password"];
  }
```

Update-form.php - Error was fixed by ensuring the reservation ID (RID) is a string.

```
if (is_string($_POST["RID"])) {
    $resID = $_POST["RID"];
  }
```

PossiblyNullArrayAccess - variable was typecasted to array data type.

staffWel.php

```
$customer = (array)$collection3->findOne(
```

login_cus.php

```
 $customer = (array)$collection->findOne(
```

update-form.php

```
$reservation = (array)$collection->findOne(
```

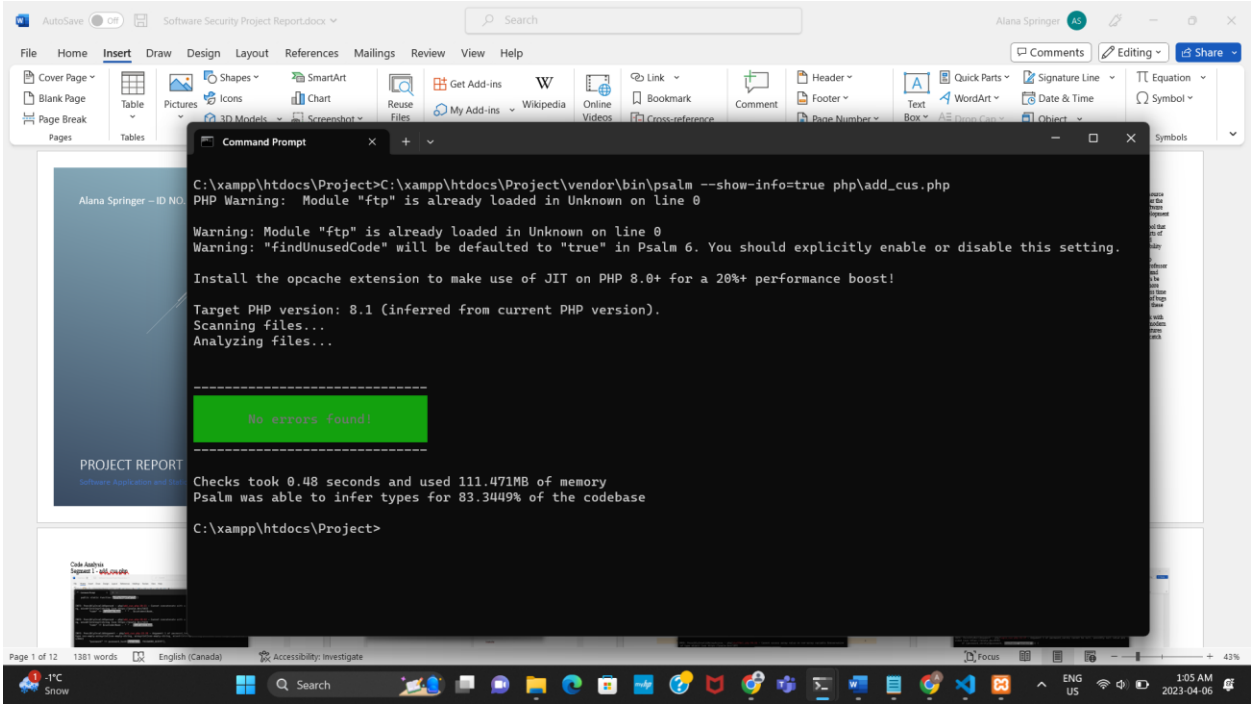
PossiblyInvalidArrayAccess – The code which caused this error was accessing an object as though it was an array, since the objects were typecasted into an array in the *PossiblyNullArrayAccess* code fix, there was no need to change the lines that psalm highlighted for this error, the errors resolved themselves.

PossiblyNullArgument - The customer array was checked to make sure it is not empty.

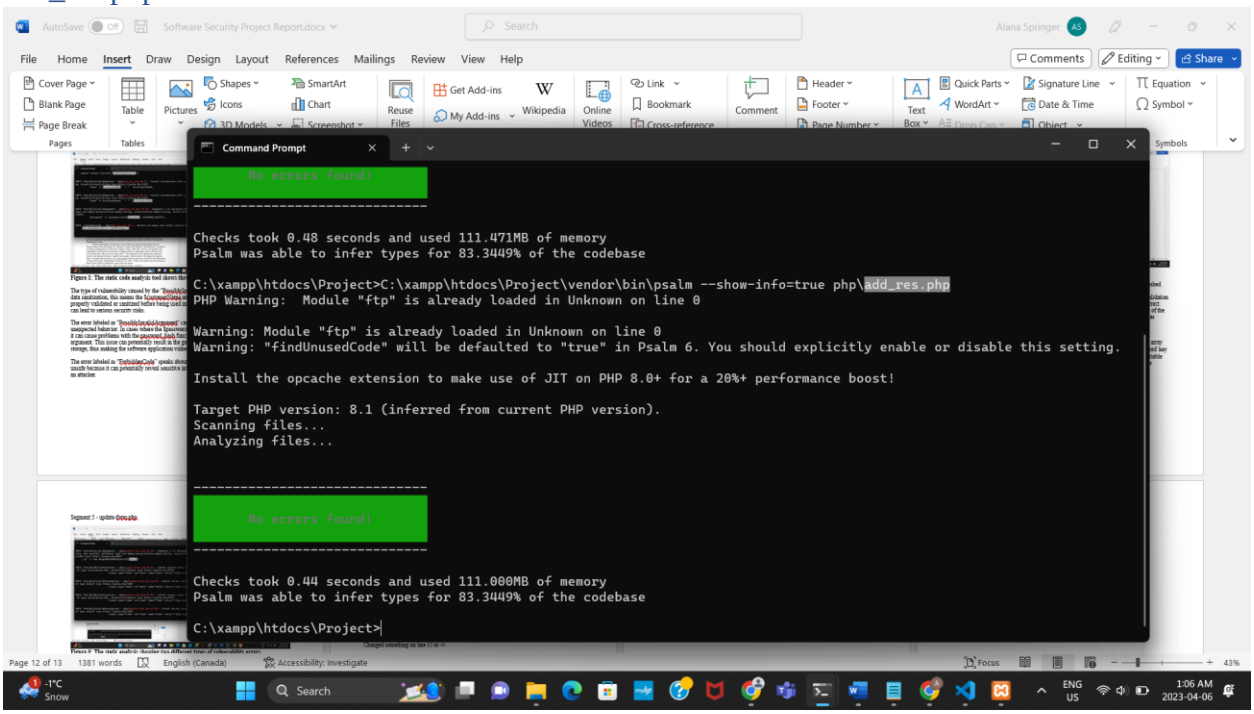
```
if ($customer != NULL) {
```

Running Psalm again to show all vulnerabilities were fixed.

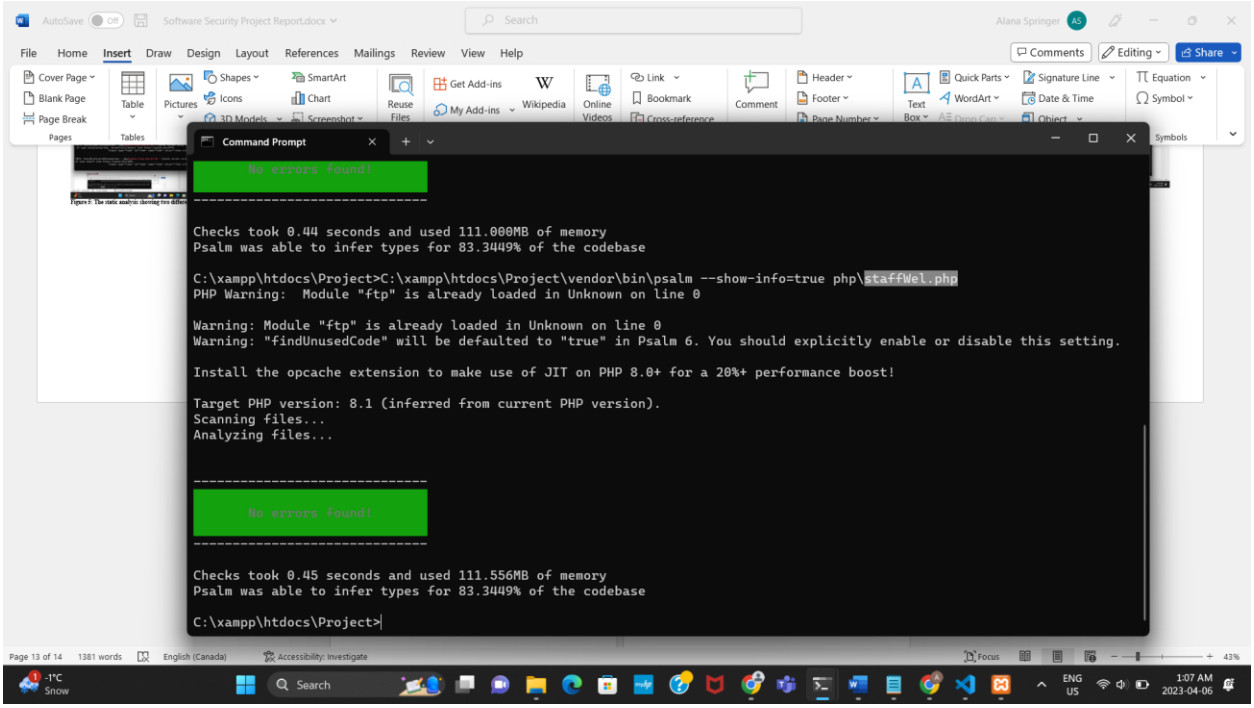
add_cus.php



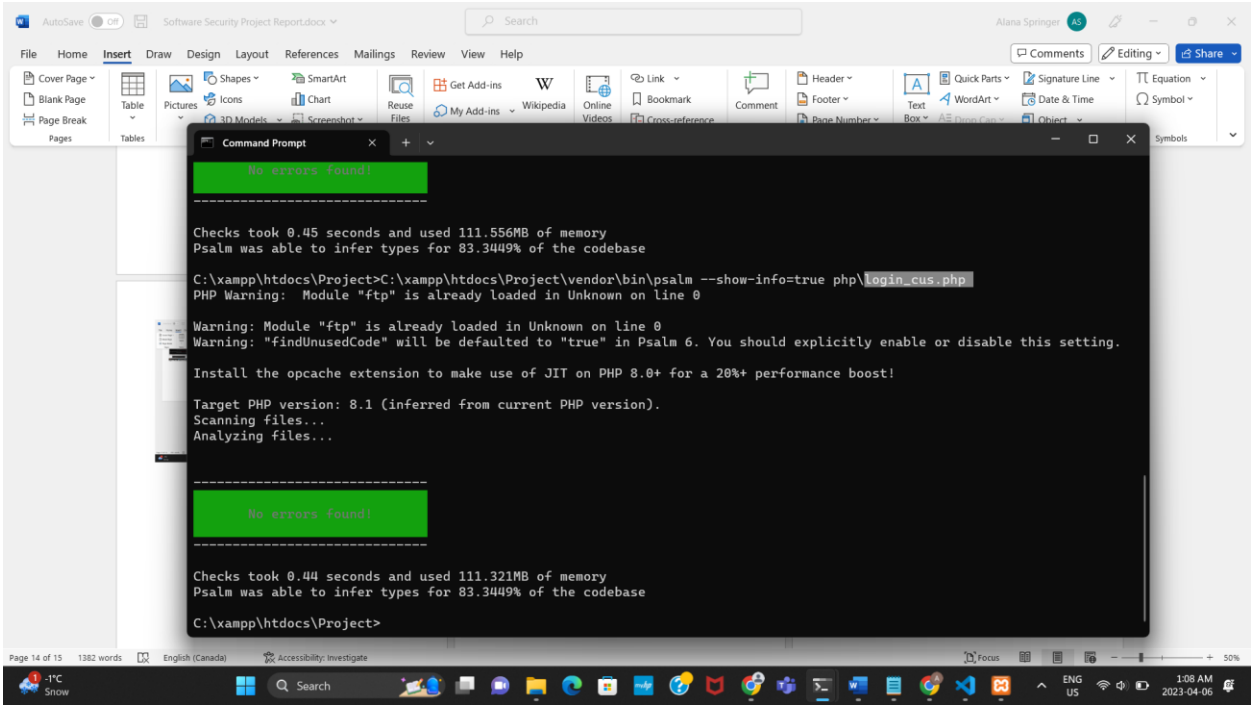
add_res.php



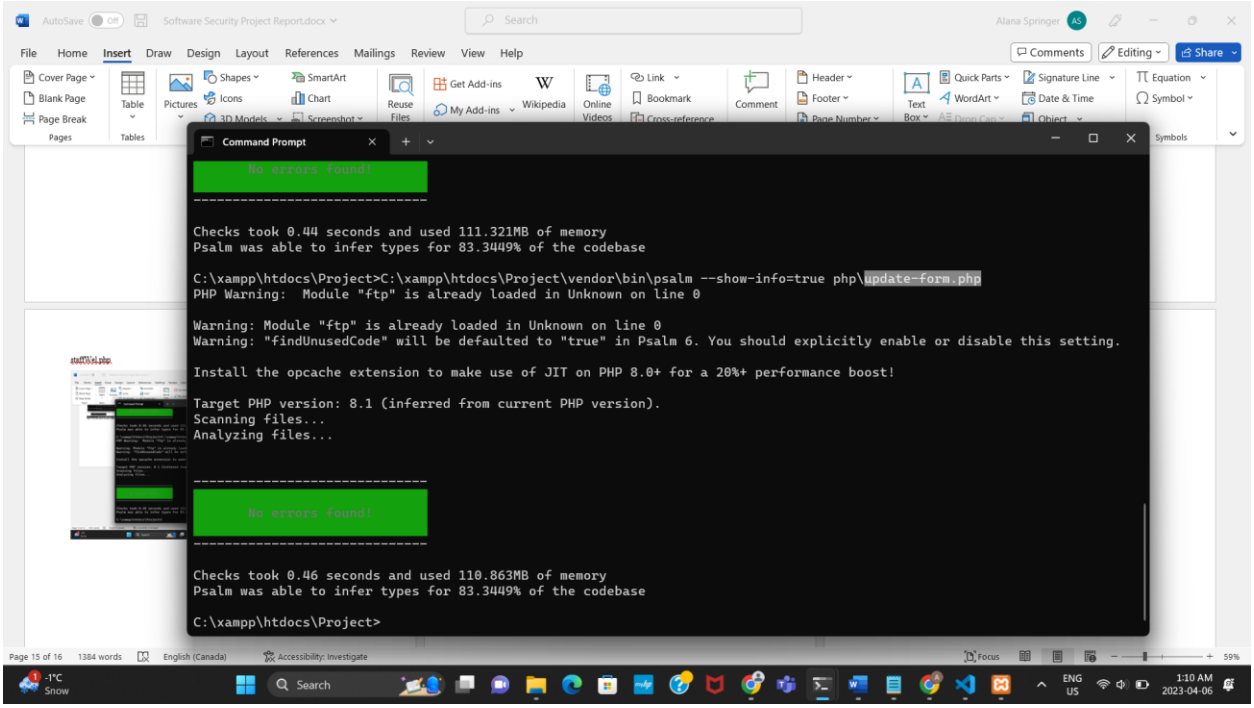
staffWel.php



Login_cus.php



update-form.php



Discussion

I gained a wealth of knowledge from this project. One of the most vital lessons I learned was the importance of keeping abreast of various software security vulnerabilities and how to address them. Another valuable takeaway was the crucial role of time management in software security projects, given the common tight deadlines in real-world software applications. Equally important was the need to allocate sufficient time for extensive testing, as rushing through it could result in software weaknesses. Therefore, it's essential to allow ample time for thorough testing to ensure optimal security. Moreover, it is important to note that one's practices eventually translate into real-world actions. Prior to undertaking this project, I had not incorporated static code analysis tools into my coding practices, as the majority of the projects I worked on were academic or personal, aimed at expanding my knowledge of programming languages. As a result, I had not typically coded with security as a top priority. However, this project served as a wake-up call, highlighting the significance of thinking like an attacker when building and testing software applications.

Pros of Psalm static analysis tool

- ≈ Psalm's analysis engine was very fast when checking code for potential errors and vulnerabilities. It scanned all the PHP files and returned over 100 vulnerabilities in seconds.
- ≈ Psalm provides clear and detailed error messages that help developers understand the problems in their code and how to fix them.
- ≈ Psalm also offers configuration options to customize the analysis, allowing developers to tailor it to their specific needs.

Cons of Psalm static analysis

- ≈ The set up was confusing and not very beginner friendly to someone who never used a static analysis tool.
- ≈ Psalm can sometimes generate false positives, which means that it reports an error when there is actually no problem with the code. This was frustrating and slowed down the development process.

References

(n.d.). Retrieved from Psalm: https://psalm.dev/

(2020). In K. Dempsey, P. Eavy, P. Moore, & E. Takamura, *Automation Support for Security: Software Vulnerability Management* (p. 105). Gaithersburg: NIST.

Dewhurst, R. (2017). *Static Code Analysis*. Retrieved from Open Worldwide Application Security Project: https://owasp.org/www-community/controls/Static_Code_Analysis

Harman, M. (2010). Why Source Code Analysis and Manipulation.

Nunes, P., Medeiros, I., Fonseca, J. C., Neves, N., Vieira, M., & Correia, M. (2018, September). *Benchmarking Static Analysis Tools for Web Security.* Retrieved from ResearchGate: https://www.researchgate.net/publication/326051525_Benchmarking_Static_Analysis_Tools_for_Web_Security

Appendix

add_cus.php

```
<?php
session_start();
$customerName = "";

if (is_string($_POST["fname"])) {
    $customerName = $_POST["fname"];
  }
  $customerLName = "";
if (is_string($_POST["lname"])) {
    $customerLName = $_POST["lname"];
  }
 $password = "";
if (is_string($_POST["password"])) {
    $password = $_POST["password"];
  }

$email = $_POST["email"];
$phone = $_POST["phone"];
$passsword2 = $_POST["conf-password"];

if ($password != $passsword2) {
    echo "Passwords don't match.";
    return;
}

require_once "../vendor/autoload.php";
$collection = (new MongoDB\Client)->bellaitalia->Customers;

try {
    $data = [
        "name" => $customerName . " " . $customerLName,
        "email" => $email,
        "phone" => $phone,
        "password" => password_hash($password, PASSWORD_BCRYPT),
    ];

    $insert = $collection->insertOne($data);
    $_SESSION['id'] = (string)$insert->getInsertedId();
    header("Location: customerWel.php");
} catch (Exception $exception) {

}
?>
```

add_res.php

```php
<?php

session_start();
$date = "";

if (is_string($_POST["date"])) {
    $date = $_POST["date"];
  }

$time = $_POST["time"];
$numSeats = $_POST["seats"];
$UserID = $_SESSION['id'];

$new_date = date('Y-m-d', strtotime($date));

require_once "../vendor/autoload.php";
$collection = (new MongoDB\Client)->bellaitalia->Reservations;

try {
    $data = [
        // "name" => $customerName . " " . $customerLName,
        // "email" => $email,
        // "phone" => $phone,
        "uid" => new MongoDB\BSON\ObjectId($UserID),
        "date" => $new_date,
        "time" => $time,
        "seats" => $numSeats,
    ];

    $insert = $collection->insertOne($data);
    header("Location: customerWel.php"); //Welcome page
} catch (Exception $exception) {

}
?>
```

staffWel.php

```
<?php
session_start();
require_once "../vendor/autoload.php";
$collection = (new MongoDB\Client)->bellaitalia->Staff;
$collection2 = (new MongoDB\Client)->bellaitalia->Reservations;
$collection3 = (new MongoDB\Client)->bellaitalia->Customers;
$UserID = $_SESSION['id'];
$cursor = $collection->findOne(
    [
        '_id' => new MongoDB\BSON\ObjectId($UserID)
    ]
);

$date = date("Y-m-d");

$resCount = $collection2->countDocuments(['date' => $date]);

$reservations = $collection2->find(['date' => $date])->toArray();

?>




<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <!-- Latest compiled and minified CSS -->
    <link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/css/bootstrap.min.css">

    <!-- jQuery library -->
    <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.3/jquery.min.js"></script>
    <script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/js/bootstrap.min.js"></script>

    <!-- Latest compiled JavaScript -->
    <script src="https://cdn.jsdelivr.net/npm/jquery@3.5.1/dist/jquery.slim.min.js"
integrity="sha384-
```

```
DfXdz2htPH0lsSSs5nCTpuj/zy4C+OGpamoFVy38MVBnE+IbbVYUew+OrCXaRkfj"
crossorigin="anonymous"></script>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@4.6.2/dist/js/bootstrap.bundle.min.js"
integrity="sha384-
Fy6S3B9q64WdZWQUiU+q4/2Lc9npb8tCaSX9FK7E8HnRr0Jz8D6OP9dO5Vg3Q9ct"
crossorigin="anonymous"></script>

    <title>Staff Home Page</title>
    <link rel="stylesheet" type="text/css" href="../css/welcome-style.css">
</head>

<body>
    <!-- NAV BAR -->
    <nav class="navbar navbar-inverse">
        <div class="container-fluid">
            <div class="navbar-header">
                <a style="font-family: Brush Script MT, Brush Script Std, cursive;" class="navbar-
brand" href="#">Bella Italia</a>
            </div>

            <ul class="nav navbar-nav navbar-right">
                <li><a href="#"><span class="glyphicon glyphicon-picture"></span>
Gallery</a></li>
                <li><a href="#"><span class="glyphicon glyphicon-info-sign"></span>
About</a></li>
                <li><a href="staff-log.html"><span class="glyphicon glyphicon-log-
out"></span>Logout</a></li>
            </ul>
        </div>
    </nav>
    <h1> Welcome
        <?php
        // echo $cursor['name'];
        ?>
    </h1>
    <div class="row">
        <h1>
            <?php
            echo $resCount;
            ?>
            Upcoming reservations</h1>
    </div>

    <div class="row middle-buttons">
        <a href="../html/make-res-staff.html">Add Reservation</a>
```

```
</div>

<table>
    <tr>
        <th>Customer Name</th>
        <th>Email</th>
        <th>Phone No</th>
        <th>Date</th>
        <th>Time</th>
        <th>Seats</th>
    </tr>
    <?php

    foreach ($reservations as $reservation) {
        if(!empty($reservation)) {
        $customer = (array)$collection3->findOne(
            [
                '_id' => new MongoDB\BSON\ObjectId($reservation['uid'])
            ]
        );

        if ($customer != NULL) {
    ?>
            <tr>
                <td><?php echo $customer['name'] ?></td>
                <td><?php echo $customer['email'] ?></td>
                <td><?php echo $customer['phone'] ?></td>
                <td><?php echo $reservation['date'] ?></td>
                <td><?php echo $reservation['time'] ?></td>
                <td><?php echo $reservation['seats'] ?></td>
            </tr>
        <?php
        } else {

        ?>
            <tr>
                <td><?php echo $reservation['name'] ?></td>
                <td><?php echo $reservation['email'] ?></td>
                <td><?php echo $reservation['phone'] ?></td>
                <td><?php echo $reservation['date'] ?></td>
                <td><?php echo $reservation['time'] ?></td>
                <td><?php echo $reservation['seats'] ?></td>
            </tr>
    <?php
        }
```

```
        }
    }

        ?>
    </table>



</body>
```

login_cus.php

```php
<?php
session_start();
$email = $_POST["email"];
$password = "";

if (is_string($_POST["password"])) {
    $password = $_POST["password"];
  }
require_once "../vendor/autoload.php";
$collection = (new MongoDB\Client)->bellaitalia->Customers;

try {
    $customer = (array)$collection->findOne(
        [
            'email' => $email
        ]
    );
    if ($customer != NULL) {
    if (password_verify($password, $customer['password'])) {
        $_SESSION['id'] = $customer['_id'];
        header("Location: customerWel.php");
    }
    else{
        echo "Email or password incorrect!";
        return;
    }
    }
} catch (Exception $exception) {

}
?>
```

update-form.php

```
<?php
session_start();
// Connect to the MongoDB server
require_once "../vendor/autoload.php";
$collection = (new MongoDB\Client)->bellaitalia->Reservations;

$resID = "";

if (is_string($_POST["RID"])) {
    $resID = $_POST["RID"];
  }
$reservation = (array)$collection->findOne(
  [
    '_id' => new MongoDB\BSON\ObjectId($resID)
  ]
);


?>


<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <!-- Latest compiled and minified CSS -->
    <link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/css/bootstrap.min.css">

    <!-- jQuery library -->
    <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.3/jquery.min.js"></script>
    <script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/js/bootstrap.min.js"></script>

    <!-- Latest compiled JavaScript -->
    <script src="https://cdn.jsdelivr.net/npm/jquery@3.5.1/dist/jquery.slim.min.js"
integrity="sha384-
DfXdz2htPH0lsSSs5nCTpuj/zy4C+OGpamoFVy38MVBnE+IbbVYUew+OrCXaRkfj"
crossorigin="anonymous"></script>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@4.6.2/dist/js/bootstrap.bundle.min.js"
integrity="sha384-
```

```
Fy6S3B9q64WdZWQUiU+q4/2Lc9npb8tCaSX9FK7E8HnRr0Jz8D6OP9dO5Vg3Q9ct"
crossorigin="anonymous"></script>

    <title>Update Page </title>
    <link rel="stylesheet" type="text/css" href="../css/welcome-style.css">
  </head>

  <body>
    <!-- NAV BAR -->
    <nav class="navbar navbar-inverse">
      <div class="container-fluid">
        <div class="navbar-header">
          <a style="font-family: Brush Script MT, Brush Script Std, cursive;" class="navbar-brand"
href="#">Bella Italia</a>
        </div>

        <ul class="nav navbar-nav navbar-right">
          <li><a href="#"><span class="glyphicon glyphicon-picture"></span> Gallery</a></li>
          <li><a href="menu.php"><span class="glyphicon glyphicon-cutlery"></span> View
Menu</a></li>
          <li><a href="#"><span class="glyphicon glyphicon-log-out"></span> Logout</a></li>
        </ul>
      </div>
    </nav>


  <div class="container-fluid formbox">
          <form action="update.php" method="post">
              <h1>Make Reservation Changes</h1>
                <div class="row">
                    <label for="date">Date</label>
                    <input type="date" id="date" name="date" value="<?php echo
$reservation['date']; ?>">
                </div>
                <div class="row">
                    <label for="time">Time</label>
                    <input type="time" id="time" name="time" value="<?php echo
$reservation['time']; ?>">
                </div>
                <div class="row">
                    <label for="seats">Number of Seats</label>
                    <input type="number" id="seats" name="seats" value="<?php echo
$reservation['seats']; ?>">
                </div>
                <br>
```

```
            <div>
            <input type="hidden" name="RID" id="RID" value="<?php echo $resID; ?>">
            <input type="submit" value="CONFIRM">
            </div>
        </form>
    </div>




</body>
</html>
```